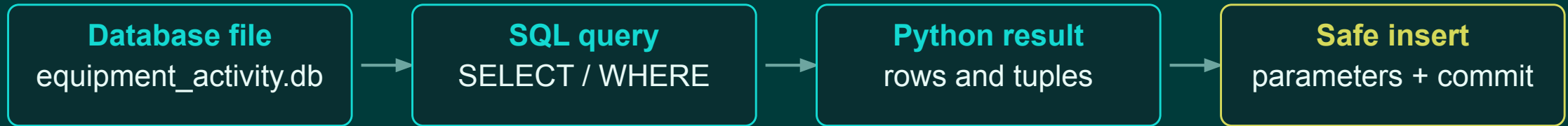


SQLite Fundamentals, SELECT, and INSERT

A one-block lesson on reading and writing local relational data

Appendix E.1

Use **SQL** to request specific rows from a SQLite database, then prepare for pandas workflows.



Mental model: SQL asks the database for the useful rows; Python receives the result.

Block Plan

30 minutes instruction + 30 minutes guided practice

Why
databases?

SQL vs
SQLite

SELECT
queries

Filtering +
sorting

INSERT
safely

Exercises

Instruction

- Relational database basics
- SELECT / WHERE / ORDER BY
- Parameterized INSERT
- commit() and NULL

Guided practice

- Connect to a database
- Read rows
- Filter and sort
- Insert and verify

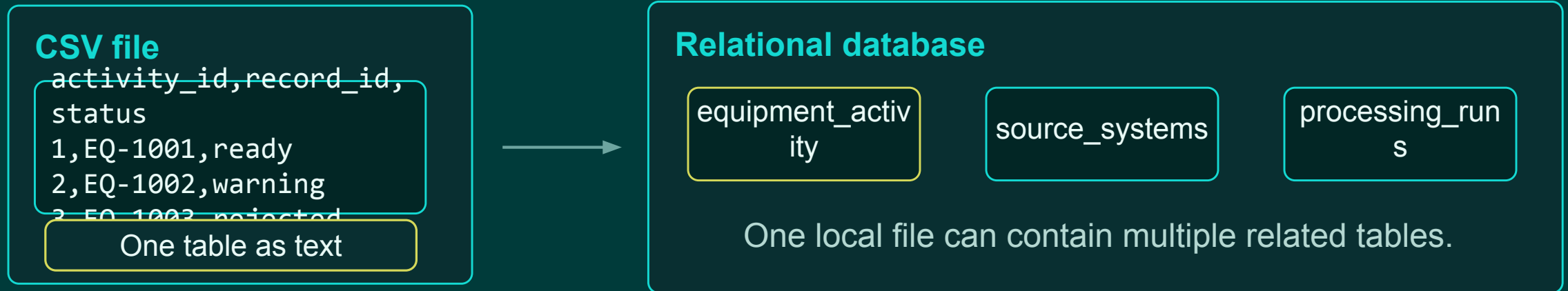
Applied output

- Notebook section
- SQLite activity intake
- Safe inserts
- Markdown explanation

Big idea: the database can select useful rows before pandas ever sees them.

From Files to Databases

A database can store structured tables and answer focused queries



Database = tables + structure + query capabilities

SQL Versus SQLite

Separate the language from the database engine

SQL

- Structured Query Language
- Used to request and modify relational data
- Shared idea across many database systems

```
SELECT record_id, status  
FROM equipment_activity  
WHERE status = 'warning';
```

SQLite

- A lightweight database engine
- Stores the database in a local file
- Works well in notebooks and small projects

```
equipment_activity.db
```

SQL is the language. SQLite is one system that understands that language.

The Example Database

We will query one activity table in a SQLite file

Database file: **equipment_activity.db**

Column	Purpose
activity_id	unique identity for the activity row
record_id	identifier from the source system
source_id	related source system identity
status	processing status
record_count	number of records represented
completion	completion percentage
error_count	number of detected errors
processed_date	date processed

Example row

```
(1, "EQ-1001", 1,
 "ready", 12000,
 100.0, 0,
 "2026-07-01")
```

Rows are records. Columns are fields. The schema defines what the table expects.

Rows, Columns, Schemas, and Keys

The database needs dependable structure

```
CREATE TABLE equipment_activity (  
    activity_id INTEGER PRIMARY KEY,  
    record_id TEXT NOT NULL,  
    source_id INTEGER,  
    status TEXT NOT NULL,  
    record_count INTEGER NOT NULL,  
    completion REAL NOT NULL,  
    error_count INTEGER NOT NULL,  
    processed_date TEXT NOT NULL  
);
```

Schema

The table structure: column names, expected types, and rules.

Primary key

A dependable identity for one row, such as activity_id.

A status like "ready" cannot identify one row because many rows can share it.

Connecting to SQLite from Python

Python includes sqlite3 in the standard library

```
import sqlite3

connection = sqlite3.connect(
    "equipment_activity.db"
)
```

Connection mental model

Python program → sqlite3 connection →
database file

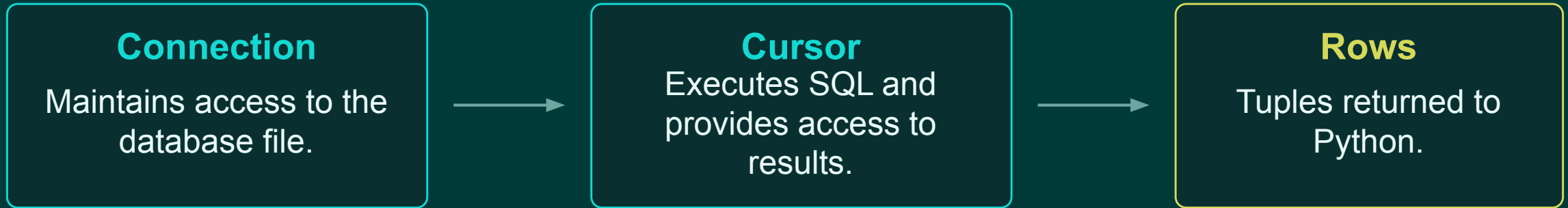
Common beginner trap

If the database path is misspelled, SQLite may create an empty database file. Then queries can fail with: no such table.

Verify the file path when a table seems to disappear.

Connections and Cursors

One object keeps access open; one object executes SQL



```
cursor = connection.execute(
    """
    SELECT *
    FROM equipment_activity
    """
)

rows = cursor.fetchall()
```


First SELECT Query

SELECT requests data from a table

```
SELECT *  
FROM equipment_activity;
```

Meaning

Request every column from the equipment_activity table.

```
cursor = connection.execute(  
    """  
    SELECT *  
    FROM equipment_activity  
    """  
)  
  
rows = cursor.fetchall()  
print(rows)
```

Typical Python result

A list of tuples. Each tuple contains values in the selected column order.

SELECT * is useful for exploration, but final queries should often request only needed columns.

Select Only the Columns Needed

Column order controls tuple order in Python

```
cursor = connection.execute(
    """
    SELECT
        record_id,
        status,
        error_count
    FROM equipment_activity
    """
)

rows = cursor.fetchall()
```

```
for record_id, status, error_count in rows:
    print(record_id, status, error_count)
```

Result tuple

record_id

status

error_count

Selected column order becomes
tuple unpacking order.

Cleaner queries make cleaner Python loops.

Filtering with WHERE

The database returns only rows where the condition is true

```
SELECT
    record_id,
    status,
    error_count
FROM equipment_activity
WHERE status = 'warning';
```

Mental model

FROM table → consider rows → keep rows
where condition is true → return selected
columns

Operator	Meaning
=	equal
!=	not equal
<	less than
>	greater than
<=	less than or equal
>=	greater than or equal

```
WHERE completion < 100
WHERE error_count > 2
WHERE processed_date >= '2026-07-10'
```

Combining Conditions

SQL conditions feel like Python Boolean logic with different syntax

```
SELECT
    record_id,
    status,
    completion,
    error_count
FROM equipment_activity
WHERE status = 'warning'
    AND error_count > 0;
```

```
SELECT
    record_id,
    status
FROM equipment_activity
WHERE status = 'warning'
    OR status = 'rejected';
```

```
Python
status == "warning" and error_count > 0
```

```
SQL
status = 'warning' AND error_count > 0
```

Important syntax difference: Python equality uses ==, SQL equality uses =.

Sorting and Limiting Results

Rows do not have a meaningful order unless the query asks for one

```
SELECT
    record_id,
    status,
    error_count
FROM equipment_activity
ORDER BY error_count DESC;
```

ORDER BY

Controls result order. DESC means largest to smallest. ASC means smallest to largest.

```
SELECT
    activity_id,
    record_id,
    processed_date
FROM equipment_activity
ORDER BY processed_date DESC
LIMIT 5;
```

LIMIT

Keeps the result small during exploration or when you only need the top rows.

Common pattern: SELECT columns → FROM table → WHERE condition → ORDER BY column → LIMIT number

Reading One Row or Many Rows

Choose the retrieval pattern that matches the result size

```
row =  
cursor.fetchone()
```

Retrieve one result row.

```
rows =  
cursor.fetchall()
```

Retrieve all remaining rows.

```
for row in cursor:  
    print(row)
```

Process rows one at a time.

Beginner guidance

`fetchall()` is fine for small classroom examples. Iterating through the cursor avoids loading every row at once for larger results.

INSERT Adds a New Row

SELECT reads data. INSERT writes data.

```
INSERT INTO equipment_activity (  
    record_id,  
    source_id,  
    status,  
    record_count,  
    completion,  
    error_count,  
    processed_date  
)  
VALUES (  
    'EQ-1010', 1, 'ready',  
    15000, 100, 0, '2026-07-10'  
);
```

Target table

equipment_activity

Listed columns

The receiving fields.

VALUES

One value per listed column.

The primary key can be omitted when SQLite is allowed to generate it.

Never Build SQL from User Values

Keep SQL syntax separate from data values

```
record_id = "EQ-1010"

sql = f"""
INSERT INTO equipment_activity (record_id)
VALUES ('{record_id}')
"""
```

Why this is risky

- Values containing quotes can break SQL
- Data becomes mixed with command syntax
- SQL injection vulnerabilities become possible



Use parameters instead.
Do not format values directly into SQL.

Parameterized SQL is not just cleaner. It is the normal safe pattern.

Parameterized INSERT

SQLite uses ? placeholders for values

```
insert_sql = """
INSERT INTO equipment_activity (
    record_id, source_id, status,
    record_count, completion,
    error_count, processed_date
)
VALUES (?, ?, ?, ?, ?, ?, ?)
"""
```

```
values = (
    "EQ-1010", 1, "ready",
    15000, 100, 0,
    "2026-07-10",
)

connection.execute(insert_sql, values)
```

SQL structure

column list + ? placeholders



Values tuple

actual data supplied separately



sqlite3

sends both safely to SQLite

The ? markers are not Python formatting. They are database placeholders.

Committing Changes

A database write is not complete until the transaction is committed

Execute INSERT

A pending database change exists

commit()

Save the change permanently

Verify

Run SELECT to confirm

```
connection.execute(insert_sql,
values)
connection.commit()
```

```
cursor = connection.execute(
    """
    SELECT record_id, status
    FROM equipment_activity
    WHERE record_id = 'EQ-1010'
    """
)
```

For introductory work: execute the INSERT, commit it, then SELECT to verify it.

Insert Multiple Rows with executemany()

Apply one SQL statement to many value tuples

```
new_rows = [  
    ("EQ-1011", 1, "ready", 9200, 100, 0, "2026-07-11"),  
    ("EQ-1012", 2, "warning", 7100, 90, 3, "2026-07-11"),  
    ("EQ-1013", 1, "ready", 6400, 100, 0, "2026-07-12"),  
]
```

```
connection.executemany(  
    insert_sql,  
    new_rows,  
)
```

```
connection.commit()
```

Course connection

Same operation → applied to multiple records.
This mirrors loops over dictionaries and DataFrames.

```
SELECT record_id, status, processed_date  
FROM equipment_activity  
ORDER BY activity_id DESC  
LIMIT 3;
```

SQL NULL and Python None

Missing values have their own comparison rules

SQL NULL

A missing or unknown SQL value.

Python None

sqlite3 can insert None as SQL NULL.

```
values = (  
    "EQ-1013",  
    None,  
    "warning",  
    5000, 80, 1,  
    "2026-07-12",  
)
```

```
WHERE source_id IS NULL  
  
WHERE source_id IS NOT NULL
```

Do not use = NULL. Use IS NULL or IS NOT NULL.

Close the Connection

Clean up when database work is complete

```
import sqlite3

connection = sqlite3.connect(
    "equipment_activity.db"
)

# Run queries here.

connection.close()
```

Simple notebook pattern

- Open the database connection
- Run the required queries
- Commit any changes
- Close the connection

For larger programs, connection management can move into a helper function later.

Guided Exercise Part 1: Read from SQLite

Connect, inspect, select, filter, sort, and limit

```
import sqlite3

connection = sqlite3.connect(
    "equipment_activity.db"
)

cursor = connection.execute(
    """
    SELECT *
    FROM equipment_activity
    LIMIT 5
    """
)
```

Tasks

- Print the first five rows
- Select record_id, status, completion, error_count
- Filter warning rows
- Filter completion below 100
- Return the five largest record_count values

Time box: 12 minutes

Compare each SQL WHERE condition with the pandas filters you already know.

Guided Exercise Part 2: Insert Safely

Use placeholders, commit, and verify

```
insert_sql = """
INSERT INTO equipment_activity (
    record_id, source_id, status,
    record_count, completion,
    error_count, processed_date
)
VALUES (?, ?, ?, ?, ?, ?, ?)
"""

values = (
    "EQ-2001", 1, "ready",
    12500, 100, 0, "2026-07-20"
)
```

Tasks

- Execute the parameterized INSERT
- Call connection.commit()
- Query the row to confirm it exists
- Use executemany() to add three rows
- Select the newest three rows

Time box: 13 minutes

Rule: no f-strings for SQL values. Use ? placeholders and pass values separately.

Applied Challenge: SQLite Activity Intake

Build a notebook section that reads, writes, verifies, and explains

Connect

Recent
rows

Warnings /
rejections

Insert
safely

executema
ny()

commit()

Verify

Close

Required notebook actions

- Select the five most recent rows
- Select warnings or rejections
- Insert one row with parameters
- Insert multiple rows with executemany()
- Query inserted rows

Markdown explanations

- What is SQL versus SQLite?
- Why are parameterized queries safer?
- Why is commit required after an insert?

Final output: a small SQLite intake notebook section, not just disconnected query cells.

Knowledge Check and Wrap-Up

Students should be ready for Appendix E.2 joins and pandas loading

What is the difference between SQL and SQLite?

What does SELECT request from the database?

Why should final queries avoid unnecessary SELECT *?

What does WHERE do?

How do ORDER BY and LIMIT shape results?

Why should INSERT values use ? placeholders?

Why is commit() required?

How do SQL NULL and Python None relate?

Final idea: SQLite stores structured data; SQL requests useful rows; Python receives the result.